So sánh các bộ phân lớp tự cài đặt trên bài toán nhận dạng chữ số viết tay MNIST

Đồ án giữa kỳ, môn Nhận dạng

Khoa Toán – Tin học, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM

Trần Trinh Nhân (24280088) Lâm Nhật Tiến (24280027)

Mã nguồn: https://github.com/TranTrinhNhan1/WriteRHand Bản chạy thử: https://trantrinhnhan1.github.io/WriteRHand/

Tóm tắt

Báo cáo so sánh bốn bộ phân lớp do nhóm tự cài đặt trên bài toán nhận dạng chữ số MNIST: Gaussian Naïve Bayes, hồi quy softmax huấn luyện bằng Gradient Descent, perceptron nhiều lớp (MLP) huấn luyện bằng lan truyền ngược, và AdaBoost-SAMME trên cây quyết định. Một mạng tích chập nhỏ (CNN) được thêm vào để so sánh với MLP. Các mô hình dùng chung một phép chia 50 000/10 000/10 000 ảnh; siêu tham số được chọn trên tập validation và tập test chỉ dùng để báo cáo. Trên tập test, độ chính xác là 81.32% (Naïve Bayes), 92.52% (softmax), 98.24% (MLP), 97.60% (AdaBoost) và 98.80% (CNN). MLP hơn AdaBoost 0.64 điểm ở seed 42, nhưng chỉ còn 0.42 điểm nếu lấy trung bình MLP trên ba seed (98.02%, độ lệch chuẩn 0.29) và AdaBoost mới được chạy một seed, nên thứ tự của hai mô hình này chưa chắc chắn; CNN cao hơn MLP ở mọi cặp seed đã chạy, với 56 714 tham số so với 535 818, và ít suy giảm hơn khi ảnh bị dịch (88.1% so với 72.9%), nhưng huấn luyện lâu hơn (143.0 s so với 11.0 s). Tăng cường dữ liệu làm tăng rõ độ bền đối với đúng loại biến đổi được dùng khi huấn luyện; tác dụng lên độ chính xác trên ảnh sạch nhỏ hơn nhiễu giữa các seed với MLP và nhất quán hơn với CNN, nhưng bị lẫn với việc tăng gấp đôi lượng dữ liệu. Trên Fashion-MNIST, MLP và CNN cho kết quả ngang nhau (88.59% và 88.49%), còn các mô hình huấn luyện trên MNIST mất từ 92.15% (CNN) xuống 49.80% (Naïve Bayes) khi đánh giá trên EMNIST Digits. Mô hình chính chỉ được huấn luyện với một seed, siêu tham số được chọn bằng quét từng tham số, và thời gian chỉ đo trên một máy CPU.

1 Giới thiệu

MNIST [10] gồm ảnh xám 28×28 của các chữ số viết tay từ 0 đến 9, đã được đưa về cùng kích thước và căn giữa. Bộ dữ liệu đủ nhỏ để huấn luyện lặp lại nhiều lần nhưng các mô hình có giả thiết rất khác nhau về cấu trúc của ảnh cho kết quả chênh nhau hơn 17 điểm phần trăm (bảng 2), nên phù hợp để so sánh có kiểm soát. Đồ án này cài đặt bốn bộ phân lớp đại diện cho bốn kiểu giả thiết: mô hình sinh với các pixel độc lập có điều kiện theo lớp (Gaussian Naïve Bayes), mô hình phân biệt tuyến tính (hồi quy softmax), mô hình phi tuyến có tham số (MLP) và ensemble tuần tự các cây quyết định (AdaBoost-SAMME). Một mạng tích chập (CNN) được thêm vào như phần mở rộng, vì nó khác MLP ở đúng một điểm: tận dụng quan hệ lân cận giữa các pixel qua trọng số dùng chung.

Báo cáo trả lời năm câu hỏi nghiên cứu:

RQ1. Bốn phương pháp bắt buộc so sánh với nhau thế nào trên MNIST dưới cùng một giao thức đánh giá?

RQ2. Độ phức tạp của mô hình ảnh hưởng thế nào đến độ chính xác, chi phí tính toán và khả năng tổng quát hóa?

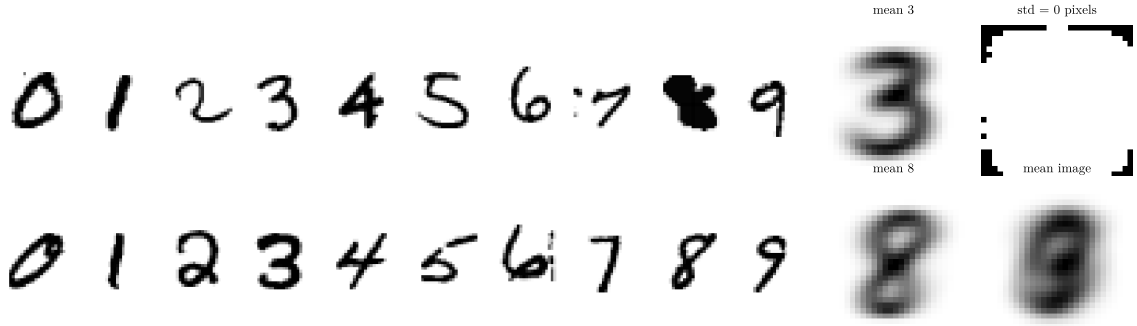
RQ3. Điều gì thay đổi khi thay MLP bằng một CNN khai thác cấu trúc không gian?

RQ4. Tăng cường dữ liệu và các nhiễu loạn có kiểm soát ảnh hưởng thế nào đến độ bền của MLP và CNN?

RQ5. Các phương pháp có nhất quán trên Fashion-MNIST và EMNIST Digits không, và khả năng tổng quát hóa giữa các bộ dữ liệu bị giới hạn ra sao?

Phạm vi tự cài đặt. Theo đề bài, phần tính toán lõi của mỗi thuật toán do nhóm viết; scikit-learn chỉ dùng để đối chiếu, PyTorch chỉ cung cấp tensor và bộ sinh số ngẫu nhiên cho MLP và CNN (không dùng `autograd`, `nn.Module`, `optim` hay các lớp tích chập có sẵn để huấn luyện). Bảng 1 liệt kê chi tiết.

Mục 2 mô tả dữ liệu, mục 3 các phương pháp, mục 4 giao thức thực nghiệm. Kết quả chính (RQ1, RQ2) ở mục 5, các thí nghiệm mở rộng (RQ3–RQ5) ở mục 6; mục 7 trả lời từng câu hỏi và mục 8 nêu các giới hạn.



Hình 1. Khảo sát dữ liệu huấn luyện. Hai hàng đầu: mẫu ngẫu nhiên của mỗi chữ số. Bên phải: ảnh trung bình của lớp 3 và lớp 8, bản đồ các pixel có phương sai bằng 0 (màu đen), và ảnh trung bình toàn tập.

2 Dữ liệu và tiền xử lý

Dữ liệu. MNIST có 60 000 ảnh huấn luyện và 10 000 ảnh kiểm thử chính thức, mỗi ảnh 28×28 điểm ảnh 8 bit (0 là nền, 255 là nét chữ), chữ số được chuẩn hóa vào hộp 20×20 và căn giữa theo trọng tâm [10]. Dữ liệu được tải bằng `torchvision.datasets.MNIST`.

Chia tập. 60 000 ảnh huấn luyện chính thức được chia phân tầng theo nhãn (seed 42) thành 50 000 ảnh train và 10 000 ảnh validation. Tập test chính thức (10 000 ảnh) được giữ riêng và chỉ dùng để báo cáo. Mã băm của phép chia (`52cc8206ff96f56d`) được lưu cùng mọi kết quả.

Chuẩn hóa. Pixel được chia cho 255 rồi chuẩn hóa $x \leftarrow (x - \bar{x})/(s + 10^{-8})$ với một cặp vô hướng $\bar{x} = 0.1306$, $s = 0.3081$ tính *chỉ trên tập train* và áp cho validation, test và bản demo. Dùng một cặp chung thay vì chuẩn hóa từng pixel giữ nguyên hình dạng của ảnh và tránh chia cho phương sai gần 0 của các pixel ở viền. Ảnh được làm phẳng thành vector 784 chiều cho các mô hình không tích chập; CNN nhận ảnh $1 \times 28 \times 28$.

Khảo sát. Các lớp gần cân bằng: lớp 1 chiếm nhiều nhất (11.24%) và lớp 5 ít nhất (9.03%), nên accuracy là chỉ số hợp lý; macro precision, recall và F1 vẫn được báo cáo để lớp ít mẫu không bị che khuất. Khoảng 81% giá trị điểm ảnh là nền, và 71 vị trí ở viền ảnh luôn bằng nền trên tập train (hình 1); chúng làm phương sai ước lượng của Naïve Bayes bằng 0 và không bao giờ được cây quyết định chọn để tách. Hệ số tương quan Pearson giữa hai pixel kề nhau theo chiều ngang, tính trong từng lớp rồi lấy trung bình trên các vị trí có phương sai khác 0, là 0.65; giả thiết độc lập có điều kiện của Naïve Bayes (mục 3.1) vì vậy không thỏa mãn trên dữ liệu này.

Khác biệt phân phối. Validation là một phần của tập huấn luyện chính thức, còn test chính thức là tập tách riêng. Cả năm mô hình đều có độ chính xác test không thấp hơn validation (bảng 2); chênh lệch là 1.65 điểm với Naïve Bayes, 0.37 điểm với softmax và không quá 0.15 điểm với ba mô hình còn lại. Chúng tôi không khảo sát nguyên nhân và chỉ dùng validation để chọn mô hình.

3 Phương pháp

Ký hiệu: $\boldsymbol{x} \in \mathbb{R}^D$ ($D = 784$) là ảnh đã chuẩn hóa và làm phẳng, $y \in \{0, \dots, K-1\}$ ($K = 10$) là nhãn, tập huấn luyện gồm N cặp $(\boldsymbol{x}_i, y_i)$. Bảng 1 nêu rõ phần nào do nhóm viết và phần nào dùng thư viện.

3.1 Gaussian Naïve Bayes

Mô hình sinh giả thiết các pixel độc lập có điều kiện theo lớp và mỗi pixel có phân phối chuẩn [2, 7]:

$$\hat{y} = \arg\max_k \Big[\log \pi_k - \tfrac{1}{2} \sum_{j=1}^{D} \Big(\log(2\pi\sigma_{kj}^2) + \frac{(x_j - \mu_{kj})^2}{\sigma_{kj}^2} \Big) \Big]. \tag{1}$$

Ước lượng hợp lý cực đại cho $\pi_k = N_k/N$, μ_{kj} và σ_{kj}^2 là trung bình và phương sai mẫu của pixel j trong lớp k. Pixel luôn bằng 0 có phương sai 0 làm $\log \sigma^2 \to -\infty$, nên mọi phương sai được cộng $\varepsilon = \texttt{var_smoothing} \cdot \max_j \mathrm{Var}(x_j)$, cùng quy ước với scikit-learn. Phép tính thực hiện trong không gian log và chuẩn hóa hậu nghiệm bằng log-sum-exp để tránh underflow khi nhân 784 thừa số. Huấn luyện cần một lượt qua dữ liệu, $O(ND)$; dự đoán một ảnh $O(KD)$. Siêu tham số duy nhất là `var_smoothing`. Giả thiết độc lập thường được dùng dù không đúng [6], nhưng ở đây nó sai rõ rệt (mục 2).

Thành phần	Nhóm tự viết (NumPy)	Thư viện
Tiền xử lý, chỉ số	Chia phân tầng, chuẩn hóa, ma trận nhầm lẫn, precision/recall/F1 macro, McNemar	`sklearn.metrics` chỉ trong kiểm thử
Naïve Bayes	Ước lượng tham số, log-likelihood, log-sum-exp	`GaussianNB` để đối chiếu
Softmax	Hàm mất mát, gradient giải tích, mini-batch GD	`LogisticRegression` để đối chiếu
MLP	Lan truyền tiến và ngược, khởi tạo He, dropout, SGD có momentum, dừng sớm	Tensor, `matmul`, bộ sinh số ngẫu nhiên của PyTorch; `autograd` và `optim` chỉ trong kiểm thử
AdaBoost-SAMME	Cập nhật SAMME, cây CART có trọng số dựng bằng histogram	`AdaBoostClassifier` để đối chiếu
CNN	im2col/col2im, tích chập, gộp cực đại, gradient của từng lớp	Như MLP; `F.conv2d`, `F.max_pool2d` chỉ trong kiểm thử

Bảng 1. Phạm vi tự cài đặt.

3.2 Hồi quy softmax (logistic đa lớp)

Với $\boldsymbol{z} = W^\top \boldsymbol{x} + \boldsymbol{b}$ và $p_k = e^{z_k} / \sum_m e^{z_m}$, hàm mất mát trên mini-batch $\mathcal{B}$ gồm cross-entropy và phạt ℓ_2 (không phạt bias):

$$L(W, \boldsymbol{b}) = -\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \log p_{y_i}(\boldsymbol{x}_i) + \frac{\lambda}{2} \|W\|_F^2. \tag{2}$$

Vì $\partial L / \partial z_k = p_k - \mathbb{1}[y = k]$, với $X \in \mathbb{R}^{B \times D}$, ma trận xác suất P và ma trận one-hot Y:

$$\nabla_W L = \tfrac{1}{B} X^\top (P - Y) + \lambda W, \qquad \nabla_{\boldsymbol{b}} L = \tfrac{1}{B} \mathbf{1}^\top (P - Y). \tag{3}$$

Softmax và cross-entropy được tính ở dạng ổn định số (trừ $\max_k z_k$, dùng $\log \sum \exp$). Cập nhật $\theta \leftarrow \theta - \eta \nabla_\theta L$ theo mini-batch có xáo trộn mỗi epoch; hàm mục tiêu lồi nên khởi tạo bằng 0. Mô hình tuyến tính nên biên quyết định giữa hai lớp là một siêu phẳng trong không gian pixel.

3.3 Perceptron nhiều lớp và lan truyền ngược

Các lớp ẩn tính $\boldsymbol{z}_l = \boldsymbol{a}_{l-1} W_l + \boldsymbol{b}_l$, $\boldsymbol{a}_l = \mathrm{ReLU}(\boldsymbol{z}_l) \odot \boldsymbol{m}_l$ với $\boldsymbol{a}_0 = \boldsymbol{x}$; lớp cuối cho logits và softmax; mất mát là cross-entropy trung bình. Trọng số khởi tạo theo He [9], $W_l \sim \mathcal{N}(0, 2/\mathrm{fan_in})$. Dropout dạng *inverted* [14] dùng mặt nạ $m_{l,j} \in \{0, 1/(1-p)\}$, chỉ bật khi huấn luyện. Lan truyền ngược [13] được viết tay:

$$\boldsymbol{\delta}_L = \tfrac{1}{B}(\mathrm{softmax}(\boldsymbol{z}_L) - Y), \quad \nabla_{W_l} = \boldsymbol{a}_{l-1}^\top \boldsymbol{\delta}_l, \quad \nabla_{\boldsymbol{b}_l} = \textstyle\sum_{\text{hàng}} \boldsymbol{\delta}_l, \quad \boldsymbol{\delta}_{l-1} = (\boldsymbol{\delta}_l W_l^\top) \odot \mathbb{1}[\boldsymbol{a}_{l-1} > 0] \odot \boldsymbol{m}_{l-1}. \tag{4}$$

Cập nhật dùng SGD có momentum [15]: $\boldsymbol{v} \leftarrow \mu \boldsymbol{v} + \boldsymbol{g}$, $\theta \leftarrow \theta - \eta \boldsymbol{v}$. Dừng sớm [12] dựa trên 8% tập train tách riêng bên trong mỗi lần huấn luyện (kiên nhẫn 5 epoch, khôi phục trọng số tốt nhất), nên tập validation không tham gia huấn luyện. Thuật toán 1 tóm tắt một epoch.

Thuật toán 1. Một epoch huấn luyện MLP (`src/models/mlp.py`)

1: Xáo trộn chỉ số của tập train, chia thành các mini-batch kích thước B.
2: Với mỗi mini-batch: lan truyền tiến, lưu $\boldsymbol{z}_l, \boldsymbol{a}_l, \boldsymbol{m}_l$ của mọi lớp; tính cross-entropy.
3: Tính $\boldsymbol{\delta}_L$ rồi lùi từ lớp cuối về lớp đầu theo công thức (4) để thu $\nabla_{W_l}, \nabla_{\boldsymbol{b}_l}$.
4: Cập nhật $\boldsymbol{v}_l \leftarrow \mu \boldsymbol{v}_l + \nabla_l$, $\theta_l \leftarrow \theta_l - \eta \boldsymbol{v}_l$.
5: Cuối epoch: tính loss trên tập dừng sớm (tắt dropout); nếu không cải thiện quá 10^{-4} trong 5 epoch thì dừng và khôi phục trọng số tốt nhất.

3.4 AdaBoost-SAMME với cây quyết định có trọng số

Boosting là ensemble thực sự: các bộ học yếu được huấn luyện tuần tự và mỗi bộ tập trung vào mẫu mà các bộ trước phân lớp sai. SAMME [17] mở rộng AdaBoost [8] sang K lớp. Khởi tạo $w_i = 1/N$; ở vòng t,

huấn luyện cây h_t với trọng số mẫu w, đặt $\mathrm{err}_t = \sum_i w_i \mathbb{1}[h_t(\boldsymbol{x}_i) \neq y_i]$ (với $\sum_i w_i = 1$) và

$$\alpha_t = \nu\Big(\ln\frac{1-\mathrm{err}_t}{\mathrm{err}_t} + \ln(K-1)\Big), \qquad w_i \leftarrow \frac{w_i\, e^{\alpha_t \mathbb{1}[h_t(\boldsymbol{x}_i)\neq y_i]}}{\sum_{i'} w_{i'} e^{\alpha_t \mathbb{1}[h_t(\boldsymbol{x}_{i'})\neq y_{i'}]}},$$
$$H(\boldsymbol{x}) = \arg\max_k \sum_t \alpha_t \mathbb{1}[h_t(\boldsymbol{x}) = k]\,, \tag{5}$$

với ν là hệ số co. Số hạng $\ln(K-1)$ chỉ yêu cầu bộ học yếu tốt hơn đoán ngẫu nhiên ($\mathrm{err} < 1 - 1/K$); vòng lặp dừng khi điều kiện này bị vi phạm. Bộ học yếu là cây CART độ sâu tối đa d giảm tạp chất Gini có trọng số [3]. Để huấn luyện được trên $50\,000 \times 784$, mỗi đặc trưng được lượng tử hóa một lần thành n_{bins} khoảng đều nhau và trọng số theo (nút, đặc trưng, bin, lớp) được cộng dồn bằng histogram, nên mọi ngưỡng của mọi đặc trưng được đánh giá trong $O(ND)$ mỗi tầng; ngưỡng thực được lưu cho dự đoán. Mỗi cây chỉ xét một tập con ngẫu nhiên các đặc trưng (`max_features`); Thuật toán 2 tóm tắt quy trình. Chi phí huấn luyện $O(T\,d\,N\,D)$ với T cây và dự đoán $O(T\,d)$; điểm số đầu ra là tỉ lệ phiếu có trọng số chứ không phải xác suất đã hiệu chỉnh.

Thuật toán 2. AdaBoost-SAMME (`src/models/adaboost.py`)

1: Lượng tử hóa X thành bin (một lần); $w_i \leftarrow 1/N$.
2: Với $t = 1, \ldots, T$: dựng cây h_t từ histogram có trọng số, chọn ở mỗi nút cặp (đặc trưng, ngưỡng) làm giảm tạp chất nhiều nhất; dừng khi không giảm hoặc đạt độ sâu d.
3: $\mathrm{err}_t \leftarrow \sum_i w_i \mathbb{1}[h_t(\boldsymbol{x}_i) \neq y_i]$; nếu $\mathrm{err}_t \geq 1 - 1/K$ thì dừng.
4: Tính α_t và cập nhật, chuẩn hóa w theo công thức (5).
5: Dự đoán: $\hat{y} = \arg\max_k \sum_t \alpha_t \mathbb{1}[h_t(\boldsymbol{x}) = k]$.

3.5 Mạng tích chập nhỏ

CNN [10] khác MLP ở chỗ mỗi đơn vị chỉ nhìn một cửa sổ cục bộ và các cửa sổ dùng chung trọng số. Cấu hình được chọn trên validation là hai khối tích chập 3×3 với 16 và 32 kênh, mỗi khối theo sau bởi ReLU và gộp cực đại 2×2, rồi hai lớp dày đặc:

$$1{\times}28{\times}28 \to 16{\times}26{\times}26 \to 16{\times}13{\times}13 \to 32{\times}11{\times}11 \to 32{\times}5{\times}5 \to 800 \to 64 \to 10.$$

Các kích thước này được đọc từ `CNN.describe()` của mô hình đã huấn luyện, tổng cộng 56 714 tham số (MLP: 535 818). Tích chập được cài bằng *im2col*: mỗi cửa sổ $k \times k \times C_{\mathrm{in}}$ được trải thành một hàng nên đầu ra là tích ma trận patches $\cdot\, W$; gradient của kernel là patches$^{\top}\, \partial L/\partial \mathrm{out}$ và gradient theo đầu vào được cộng dồn bằng *col2im*. Gộp cực đại ghi lại vị trí cực đại để chuyển gradient về đúng ô. Huấn luyện dùng cùng SGD momentum, khởi tạo He, dropout và dừng sớm như MLP.

4 Giao thức thực nghiệm

Chọn siêu tham số. Bảng lưới đầy đủ cho MLP có hàng trăm tổ hợp, quá tốn kém trong thời gian của đồ án, nên chúng tôi dùng một vòng *quét từng siêu tham số*: xuất phát từ cấu hình mặc định, lần lượt quét mỗi siêu tham số trong khi giữ các tham số khác ở giá trị tốt nhất hiện có, và chọn giá trị cho validation accuracy cao nhất (hòa thì chọn giá trị đứng trước). Chi phí là tổng chứ không phải tích số lượng giá trị, và mỗi lượt quét chính là đồ thị "đổi một tham số, giữ nguyên phần còn lại" (hình 3). Công cụ này do nhóm tự viết (`src/experiments/search.py`); nhóm không dùng Optuna hay thư viện tối ưu siêu tham số. Số vòng boosting và số epoch của softmax được chọn từ đường cong theo vòng của một lần huấn luyện dài. Hàm tìm kiếm không nhận mảng test. Cách làm này không xét tương tác giữa các siêu tham số và kết quả phụ thuộc vào thứ tự quét; với AdaBoost và CNN, một số giá trị tốt nhất nằm ở mép của khoảng đã quét (mục 8). Cấu hình được chọn của từng mô hình có trong bảng 10.

Huấn luyện cuối và các chỉ số. Cấu hình được chọn được huấn luyện lại trên 50 000 ảnh train với seed 42, rồi báo cáo accuracy, macro precision, macro recall và macro F1 cùng ma trận nhầm lẫn trên cả ba tập. Với ma trận nhầm lẫn C (hàng là nhãn thật), $\mathrm{TP}_k = C_{kk}$, $\mathrm{FP}_k = \sum_i C_{ik} - \mathrm{TP}_k$, $\mathrm{FN}_k = \sum_j C_{kj} - \mathrm{TP}_k$; precision, recall và F_1 được tính theo từng lớp rồi lấy trung bình không trọng số.

So sánh có kiểm định. Hai mô hình được đánh giá trên cùng 10 000 ảnh test được so sánh bằng kiểm định McNemar chính xác: với b ảnh chỉ mô hình thứ nhất đúng và c ảnh chỉ mô hình thứ hai đúng, p là xác suất hai phía của nhị thức $\mathrm{Bin}(b + c, \frac{1}{2})$ [5]. Kiểm định này chỉ phản ánh sai số do lấy mẫu tập test

Mô hình	Train	Val	Test	P_{macro}	R_{macro}	$F1_{macro}$
Gaussian NB	80.13	79.67	**81.32**	82.98	80.95	81.16
Softmax regression	93.62	92.15	**92.52**	92.45	92.41	92.42
MLP	99.87	98.18	**98.24**	98.23	98.23	98.23
AdaBoost-SAMME	100.00	97.59	**97.60**	97.58	97.58	97.58
CNN	99.40	98.65	**98.80**	98.79	98.79	98.79

Bảng 2. Accuracy (%) trên train, validation và test, cùng precision, recall và F1 macro (%) trên test. Một lần huấn luyện (seed 42) cho mỗi mô hình.

của *hai mô hình đã huấn luyện cụ thể*, không phản ánh biến thiên do seed huấn luyện. Vì vậy chúng tôi báo cáo thêm kết quả nhiều seed ở những nơi có.

Đo thời gian. Thời gian đo bằng `time.perf_counter` trên CPU, không dùng GPU (13th Gen Intel Core i7-13620H, 16 luồng logic, PyTorch 8 luồng, Python 3.12.14). Thời gian huấn luyện là thời gian chạy của `fit` trên 50 000 ảnh; thời gian dự đoán là trung vị của 5 lần dự đoán 10 000 ảnh test và độ trễ là trung vị của 200 lần dự đoán một ảnh. Các lượt chạy có thể trùng với công việc khác trên cùng máy: cùng cấu hình và seed, CNN mất 143.0 s ở lần huấn luyện cuối nhưng 266 s và 312 s ở hai lần chạy trong thí nghiệm tăng cường, nên thời gian chỉ nên được so ở mức bậc độ lớn.

Đối chiếu với scikit-learn. Với Naïve Bayes, softmax, MLP và AdaBoost, scikit-learn được huấn luyện với *cùng siêu tham số* và cùng dữ liệu, không chọn lại siêu tham số, và đánh giá trên cùng tập test. Đây là kiểm tra tính đúng đắn của cài đặt, không phải cuộc đua giữa hai thư viện. Ngoài ra bộ kiểm thử tự động (107 kiểm thử, `pytest`) kiểm tra: tham số, xác suất và log-xác suất của Naïve Bayes khớp `GaussianNB` (sai số $< 10^{-6}$); gradient của softmax, MLP và CNN khớp sai phân hữu hạn trung tâm; gradient của MLP và CNN khớp autograd của PyTorch trên mạng nhỏ độ chính xác kép; bước SGD momentum khớp `torch.optim.SGD`; tích chập im2col khớp một vòng lặp ngây thơ và `F.conv2d`; cây một nút và nhiều tầng cho cùng phân hoạch với `DecisionTreeClassifier` có `sample_weight` trên dữ liệu lưới nguyên; phép chia phân tầng, chuẩn hóa và các chỉ số khớp scikit-learn; và mô hình xuất ONNX cho cùng nhãn với mô hình Python trên toàn bộ ảnh test.

Tái lập và theo dõi. Mọi kết quả được ghi thành tệp JSON do chương trình sinh (`results/`); các bảng và đoạn LaTeX của báo cáo được sinh từ các tệp đó nên không có số liệu nhập tay. Các lần chạy được ghi bằng Weights & Biases [1] ở chế độ offline (nhóm chưa dùng khóa API), kèm tên mô hình, siêu tham số, seed, mã commit và mã băm phép chia. Lệnh tái lập nằm ở mục D; số thập phân trong báo cáo dùng dấu chấm.

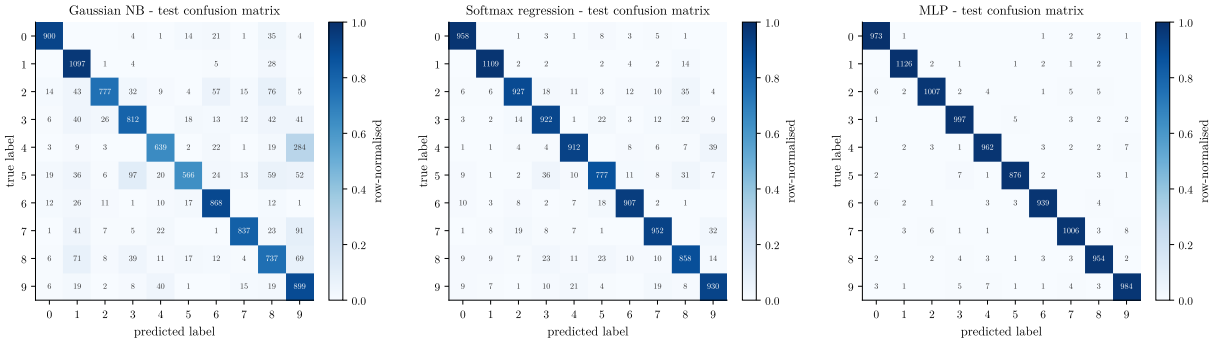
5 Kết quả chính trên MNIST

Mọi số liệu trong mục này được đọc từ các tệp kết quả do chương trình ghi (thư mục `results/`: `final`, `tuning`, `baselines`).

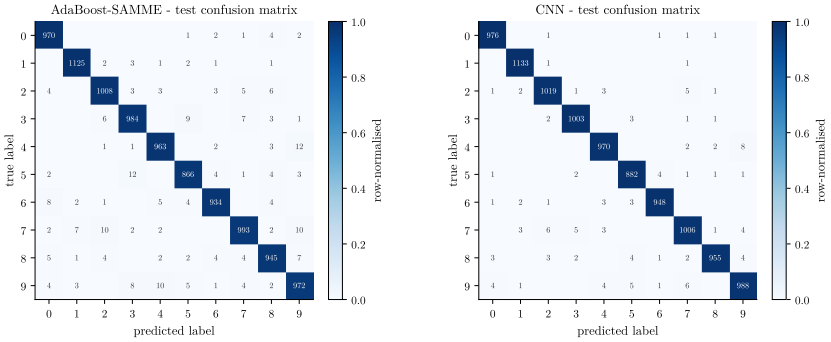
5.1 So sánh độ chính xác (RQ1)

Thứ tự theo độ chính xác trên test là CNN (98.80%), MLP (98.24%), AdaBoost (97.60%), softmax (92.52%) và Naïve Bayes (81.32%) (bảng 2). Macro-F1 lệch accuracy không quá 0.16 điểm ở cả năm mô hình, phù hợp với việc các lớp gần cân bằng. Với $n = 10\,000$, khoảng tin cậy nhị thức 95% của một accuracy gần 98% rộng khoảng ± 0.3 điểm, nên các khoảng cách dưới một điểm cần kiểm định ghép cặp. Kiểm định McNemar cho cặp MLP–AdaBoost cho 127 ảnh chỉ MLP đúng và 63 ảnh chỉ AdaBoost đúng ($p = 4 \times 10^{-6}$), và cho cặp CNN–MLP cho 120 ảnh chỉ CNN đúng và 64 ảnh chỉ MLP đúng ($p = 4.4 \times 10^{-5}$). Chúng tôi đã tính lại cả hai giá trị p từ các tệp dự đoán và thu được kết quả trùng khớp.

Tuy nhiên McNemar chỉ cho biết các khác biệt này khó xảy ra do sai số lấy mẫu của tập test với đúng hai mô hình đã huấn luyện. Biến thiên giữa các seed được đo ở thí nghiệm tăng cường (mục 6.2), nơi điều kiện không tăng cường dùng đúng cấu hình ở đây: MLP đạt 98.24, 98.14 và 97.69% với seed 42, 43, 44 (trung bình 98.02%, độ lệch chuẩn mẫu 0.29), CNN đạt 98.80 và 98.84% với seed 42 và 43. Do đó CNN cao hơn MLP ở mọi cặp seed đã chạy, còn khoảng cách 0.64 điểm giữa MLP và AdaBoost ở seed 42 thu còn 0.42 điểm khi so với trung bình ba seed của MLP. AdaBoost chỉ được chạy một seed và có tính ngẫu nhiên riêng (chọn tập con đặc trưng cho mỗi cây) chưa được đo, nên chúng tôi không kết luận về thứ tự MLP và AdaBoost. Khoảng cách giữa hai mô hình tuyến tính/sinh và ba mô hình còn lại (hơn 5 điểm) lớn hơn mọi



(a) Naïve Bayes (b) Softmax (c) MLP



(d) AdaBoost (e) CNN

Hình 2. Ma trận nhầm lẫn trên tập test (hàng: chữ số thật; cột: chữ số dự đoán; số trong ô là số ảnh, màu theo tỉ lệ của hàng).

nguồn nhiễu đã quan sát.

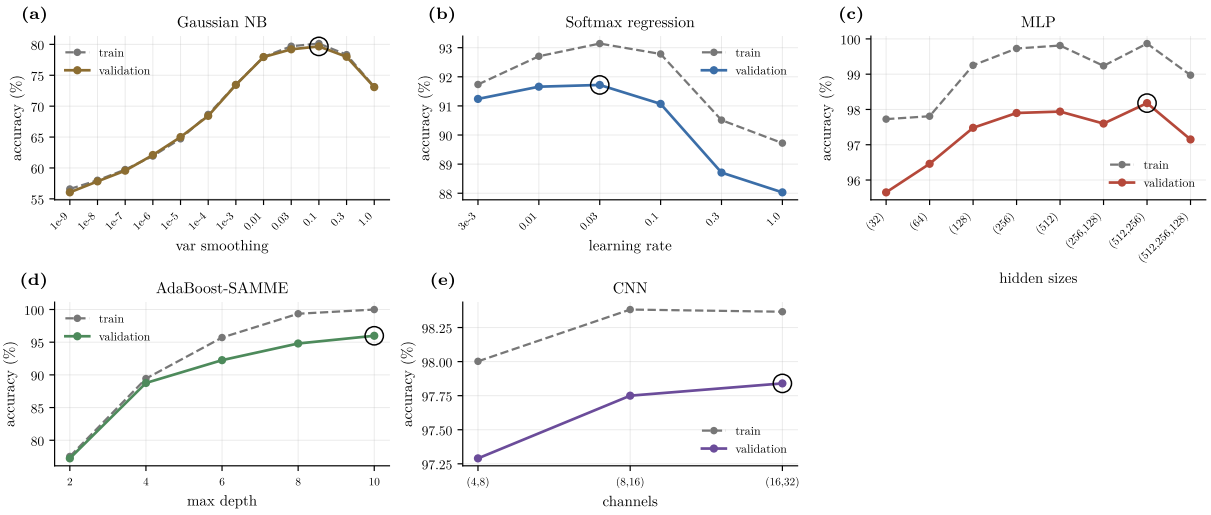
5.2 Các lỗi phân lớp

Hình 2 cho thấy cả năm mô hình đều nhầm nhiều nhất những cặp có nét chung. Cặp $4 \to 9$ là cặp nhầm lớn nhất (hoặc đồng hạng) của Naïve Bayes (284 trong 1868 lỗi), softmax (39 trong 748), AdaBoost (12 trong 240, đồng hạng với $5 \to 3$) và CNN (8 trong 120); với MLP cặp lớn nhất là $7 \to 9$ (8 trong 176 lỗi). Các cặp $5 \to 3$ và $7 \to 9$ cũng nằm trong ba cặp đầu của ít nhất ba mô hình. Với Naïve Bayes, 547 trong 1 868 lỗi (29%) là các ảnh bị dự đoán thành chữ số 9 và 313 ảnh bị dự đoán thành 8; chúng tôi chưa kiểm tra nguyên nhân của sự lệch về hai lớp này.

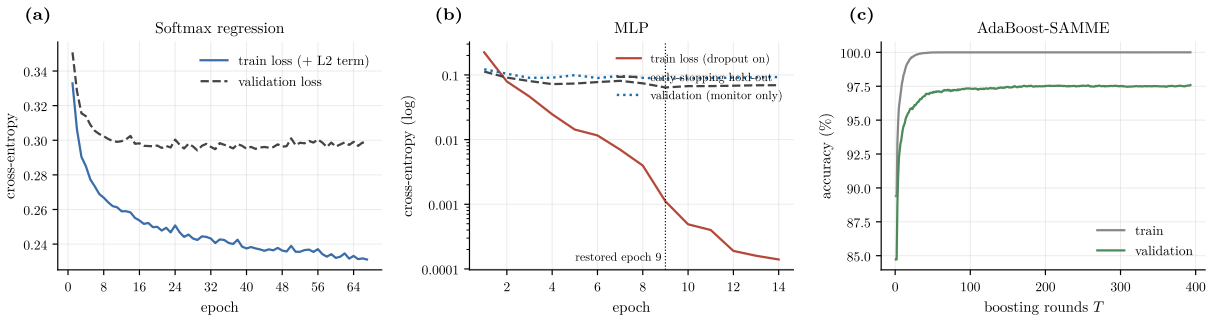
5.3 Siêu tham số và độ phức tạp (RQ2)

Hình 3 cho thấy bốn kiểu phản ứng với dung lượng hoặc mức điều chuẩn. (a) Với Naïve Bayes, validation accuracy tăng từ 56.05% ở `var_smoothing` $= 10^{-9}$ lên 79.67% ở 0.1 rồi giảm khi làm trơn quá mức; train và validation gần như trùng nhau, nên mô hình bị giới hạn bởi giả thiết chứ không bởi overfitting. (b) Với softmax, tốc độ học 0.03 cho validation cao nhất và các giá trị lớn hơn làm giảm cả train lẫn validation, tức là hiện tượng tối ưu hóa chứ không phải overfitting. (c) Với MLP, mạng hai lớp ẩn $(512, 256)$ đạt 98.18%, cao hơn mạng một lớp ẩn 512 (97.94%) và mạng ba lớp ẩn $(512, 256, 128)$ (97.15%); vì số epoch và tốc độ học được giữ cố định trong lượt quét này nên chúng tôi không tách được ảnh hưởng của độ sâu khỏi ảnh hưởng của việc tối ưu. (d) Với AdaBoost, độ sâu cây là tham số mạnh nhất: validation tăng đơn điệu từ 77.23% (độ sâu 2) đến 95.98% (độ sâu 10) ở 50 vòng, trong khi train accuracy đạt 99.99% ở độ sâu 10. (e) Với CNN, thêm một lớp dày đặc nâng validation từ 97.84 lên 98.57%, nhiều hơn việc tăng số kênh ($97.29 \to 97.84\%$); một giá trị tốc độ học (0.1) làm huấn luyện phân kỳ (11.24%, tức đoán ngẫu nhiên).

Đường cong học (hình 4) cho thấy softmax có loss validation gần như phẳng sau khoảng 15 epoch (cực tiểu ở epoch 29) trong khi loss train tiếp tục giảm, MLP dừng sớm ở epoch `14` và khôi phục epoch `9`, và AdaBoost đạt 100% trên train từ rất sớm trong khi validation tăng chậm tới 97.59% ở `393` vòng. Điều này nhất quán với việc sai số huấn luyện bằng 0 không đồng nghĩa với hết khả năng tổng quát hóa của boosting. Hình 5(c) cho thấy hiệu số train $-$ test lớn nhất ở AdaBoost (+2.40 điểm) và MLP (+1.63), nhỏ nhất ở CNN (+0.60), còn Naïve Bayes có test cao hơn train (-1.19). Số tham số không tăng đơn điệu theo



Hình 3. Accuracy train và validation theo siêu tham số có ảnh hưởng lớn nhất của từng mô hình (xem thêm mục A). Vòng tròn đánh dấu giá trị được chọn. Các lượt quét chạy tuần tự nên phần còn lại của cấu hình ở giá trị tốt nhất tìm được trước đó.



Hình 4. (a) Softmax: loss train và validation theo epoch. (b) MLP: loss train (có dropout nếu bật), loss trên tập dừng sớm và loss validation chỉ để theo dõi; đường chấm đứng đánh dấu epoch được khôi phục. (c) AdaBoost: accuracy theo số vòng boosting.

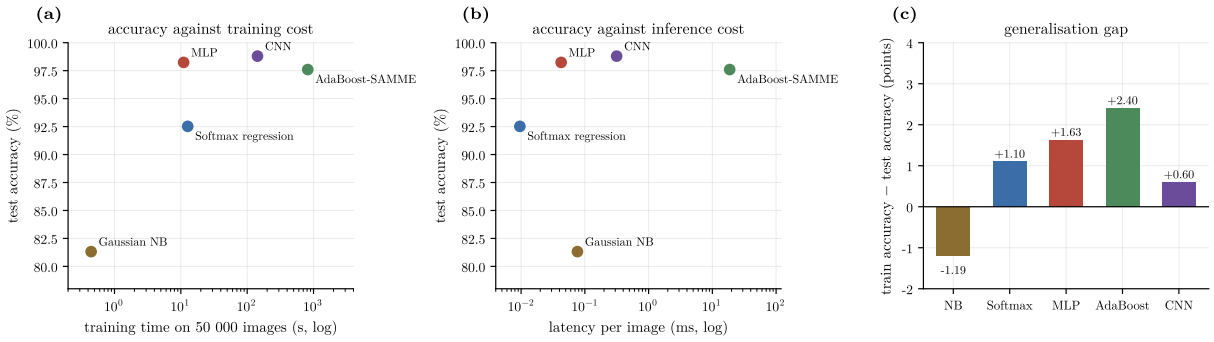
độ chính xác: CNN có 56 714 tham số, ít hơn MLP gần mười lần nhưng chính xác hơn, và Naïve Bayes lưu 15 690 số nhưng kém xa softmax với 7 850 tham số. Điều quyết định là giả thiết cấu trúc của mô hình chứ không chỉ kích thước của nó.

5.4 Chi phí tính toán

AdaBoost đắt nhất ở cả hai trục (bảng 3, hình 5): 817.9 s huấn luyện, hơn bảy mươi lần MLP, và 18.705 ms cho một ảnh. Độ trễ này phản ánh cả cách cài đặt: bộ dự đoán duyệt 393 cây bằng một vòng lặp Python, nên con số không chỉ xuất phát từ độ phức tạp $O(Td)$ của thuật toán. MLP nhanh hơn CNN cả khi huấn luyện (11.0 s so với 143.0 s) lẫn khi dự đoán (0.042 ms so với 0.315 ms); với số epoch tương đương (14 và 13), hiệu số này đến từ chi phí mỗi epoch của tích chập, không phải từ ngân sách huấn luyện khác nhau. Softmax và Naïve Bayes rẻ nhất, nhưng softmax cần 12.7 s vì lặp 67 epoch trên toàn bộ dữ liệu trong khi

Mô hình	Huấn luyện (s)	Dự đoán 10 000 ảnh (s)	Một ảnh (ms)	Tệp xuất (KiB)
Gaussian NB	0.4	0.226	0.076	62
Softmax regression	12.7	0.024	0.010	31
MLP	11.0	0.051	0.042	2 094
AdaBoost-SAMME	817.9	0.751	18.705	9 457
CNN	143.0	1.163	0.315	223

Bảng 3. Chi phí trên CPU (cấu hình ở mục 4). Huấn luyện trên 50 000 ảnh; dự đoán 10 000 ảnh test; thời gian một ảnh là trung vị của 200 lần dự đoán. Tệp xuất là ONNX (Naïve Bayes, softmax, MLP, CNN) hoặc tệp nhị phân của các cây (AdaBoost).



Hình 5. Accuracy test theo thời gian huấn luyện (a) và thời gian dự đoán một ảnh (b), thang log; (c) hiệu số accuracy train và test.

Mô hình	Test (ours)	Test (sklearn)	$\lvert\Delta\rvert$	Fit ours (s)	Fit sklearn (s)
Gaussian NB	81.32	81.32	0.00	0.4	0.2
Softmax regression	92.52	91.86	0.66	12.7	154.6
MLP	98.24	98.02	0.22	11.0	57.8
AdaBoost-SAMME	97.60	97.50	0.10	817.9	1892.8

Bảng 4. Bản tự cài đặt và scikit-learn với cùng siêu tham số: accuracy test (%), hiệu tuyệt đối và thời gian huấn luyện.

Naïve Bayes chỉ cần một lượt (0.4 s). Kích thước tệp xuất của AdaBoost (9 457 KiB) cũng lớn nhất.

5.5 Đối chiếu với scikit-learn

Naïve Bayes trùng khớp hoàn toàn với scikit-learn (bảng 4). Ba mô hình còn lại lệch từ 0.10 đến 0.66 điểm: softmax của nhóm cao hơn 0.66 điểm vì scikit-learn chạy L-BFGS tới giới hạn 1000 vòng không hội tụ và không dừng sớm, còn nhóm dừng ở epoch chọn bằng validation; MLP chênh 0.22 điểm, nhỏ hơn độ lệch chuẩn giữa các seed (0.29); AdaBoost chênh 0.10 điểm nhưng hai bản không dùng cùng thuật toán chia nút (nhóm dùng histogram 32 bin và nửa số đặc trưng cho mỗi cây, scikit-learn tìm ngưỡng chính xác trên mọi đặc trưng), nên thời gian không so sánh sạch được. Kết quả cho thấy các cài đặt khớp với thư viện trong phạm vi các khác biệt đã giải thích, không cho thấy ưu thế của bên nào.

6 Các thí nghiệm mở rộng

Các thí nghiệm dưới đây dùng cùng mã, cùng giao thức và cùng bộ đánh giá như mục 5. Cấu hình của MLP và CNN là cấu hình đã chọn trên MNIST; các bảng chính ở mục 5 không dùng tăng cường dữ liệu.

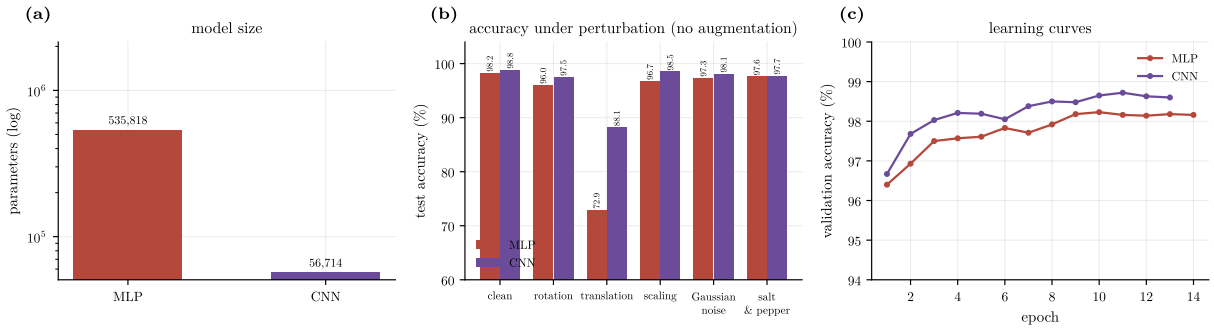
6.1 CNN so với MLP (RQ3)

CNN được tinh chỉnh bằng cùng quy trình quét từng tham số trên validation (10 cấu hình, so với 22 của MLP; ngân sách tinh chỉnh vì vậy không bằng nhau). Bảng 5 và hình 6 so sánh hai mô hình trên cùng 10 000 ảnh test.

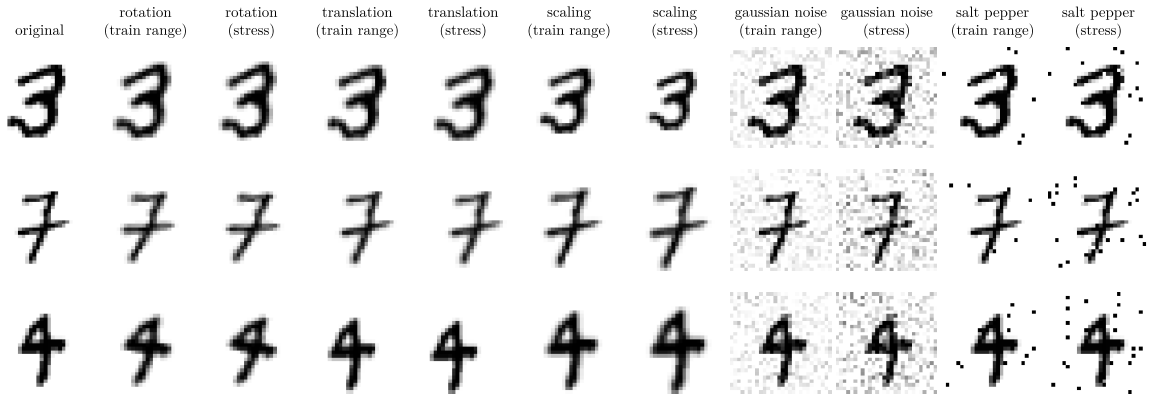
Trên ảnh sạch, CNN cao hơn MLP 0.56 điểm với seed 42 (98.80% so với 98.24%) và 0.8 điểm khi so với trung bình ba seed của MLP; với số tham số ít hơn 9.4 lần, hiệu số train − test cũng nhỏ hơn (0.60

	MLP	CNN
Số tham số	535 818	56 714
Accuracy test (%)	98.24	98.80
Accuracy train − test (điểm)	1.63	0.60
Thời gian huấn luyện (s)	11	143
Độ trễ một ảnh (ms)	0.042	0.315

Bảng 5. CNN và MLP trên MNIST, cùng giao thức; mọi số lấy từ các lần chạy đã ghi. Accuracy khi ảnh test bị biến đổi nằm ở hình 6b. Kiểm định McNemar chính xác trên dự đoán test: chỉ MLP đúng 64 ảnh, chỉ CNN đúng 120 ảnh, $p = 4.4e-05$.



Hình 6. CNN và MLP: số tham số (a), accuracy khi ảnh test bị biến đổi cố định, huấn luyện không tăng cường (b), và validation accuracy theo epoch (c).



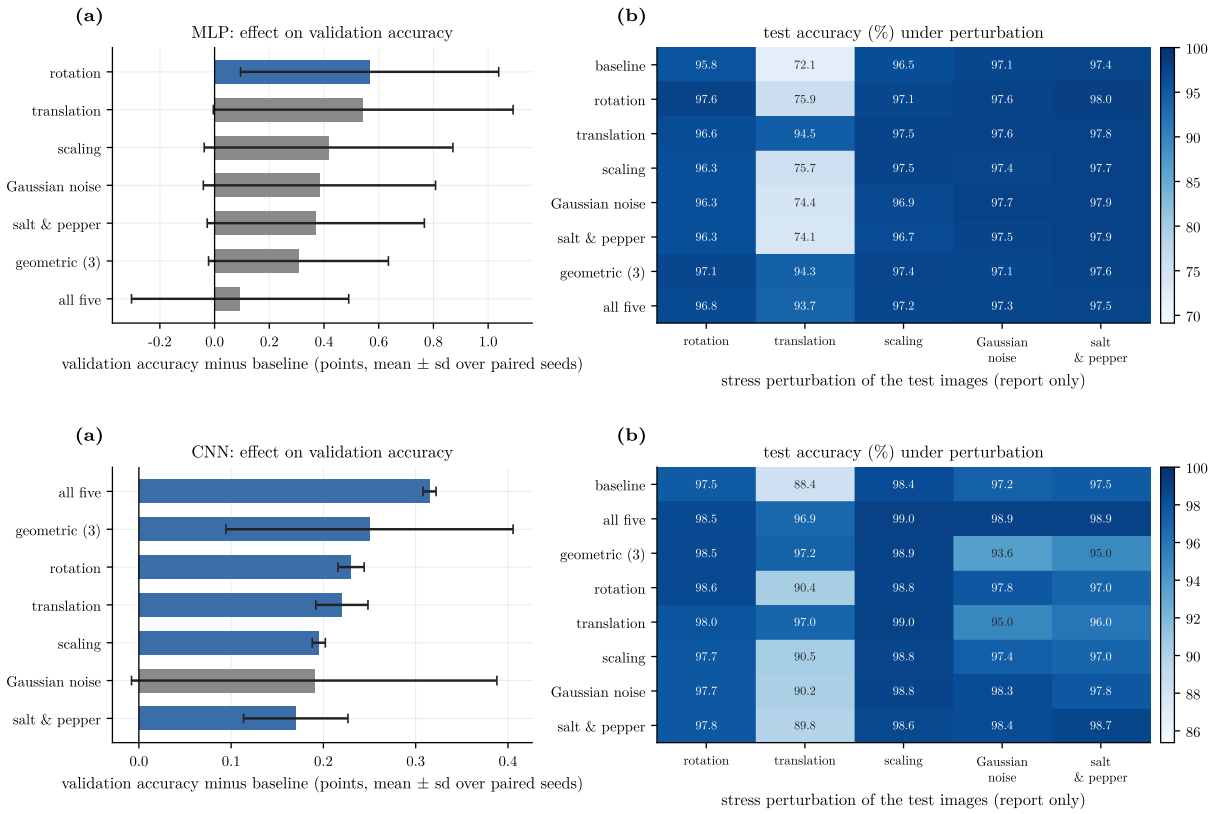
Hình 7. Ảnh gốc và từng phép biến đổi ở mức huấn luyện và mức kiểm tra độ bền (cùng các ảnh nguồn).

so với 1.63 điểm). Khác biệt lớn hơn nhiều khi ảnh test bị biến đổi (hình 6b): sau khi dịch 3 pixel, MLP còn 72.86% còn CNN 88.14%; với xoay, co giãn và nhiễu Gauss khoảng cách là 1.5, 1.8 và 0.8 điểm, và với nhiễu muối tiêu gần như không có (97.71 so với 97.56%). Mẫu hình này nhất quán với việc trọng số dùng chung và gộp cục bộ làm đầu ra ít nhạy với dịch chuyển nhỏ, nhưng chúng tôi không có thí nghiệm tách riêng từng thành phần (ví dụ CNN không gộp), nên đây là giả thuyết chứ không phải kết luận nhân quả. Cái giá là thời gian: huấn luyện lâu hơn khoảng 13 lần và mỗi ảnh chậm hơn khoảng 7 lần (bảng 3), trong cùng một cài đặt NumPy trên CPU.

6.2 Tăng cường dữ liệu và độ bền (RQ4)

Thiết kế. Năm phép biến đổi chỉ áp dụng cho tập train, nhãn không đổi (hình 7): xoay đều trong $[-15°, 15°]$; dịch đều trong $[-2, 2]$ pixel theo mỗi trục; co giãn đều trong $[0.9, 1.1]$; nhiễu Gauss $\sigma = 0.1$ trên thang $[0, 1]$; nhiễu muối tiêu trên 2% số pixel. Các phép hình học được gộp thành một phép affine duy nhất cho mỗi ảnh (nội suy song tuyến tính). Mỗi điều kiện giữ nguyên siêu tham số, seed và quy tắc dừng sớm, chỉ đổi tập train: thêm một bản sao biến đổi của mỗi ảnh (100 000 ảnh). Có tám điều kiện (gốc, từng phép riêng lẻ, nhóm hình học, cả năm phép), chạy với 3 seed (MLP) hoặc 2 seed (CNN) và so sánh ghép cặp theo seed. Điều kiện được chọn chỉ theo validation accuracy trung bình. Độ bền được đo trên tập test được biến đổi một lần với seed cố định ở mức mạnh hơn lúc huấn luyện (xoay 20°, dịch 3 pixel, co giãn 0.8–1.2, $\sigma = 0.2$, 4% muối tiêu) và chỉ dùng để báo cáo. Lưu ý rằng mọi điều kiện tăng cường có dữ liệu gấp đôi điều kiện gốc, nên ảnh hưởng của *loại biến đổi* và của *lượng dữ liệu hay số bước cập nhật* bị lẫn; không có điều kiện đối chứng nào sao chép ảnh gốc không biến đổi.

MLP. Mọi điều kiện cho validation accuracy cao hơn điều kiện gốc 0.09 đến 0.57 điểm, nhưng độ lệch chuẩn của hiệu ghép cặp (0.33–0.55 điểm) cùng bậc với chính mức tăng, nên với ba seed chúng tôi không khẳng định phép biến đổi nào cải thiện độ chính xác trên ảnh sạch. Điều kiện "xoay" có validation cao nhất và được chọn. Ngược lại, tác dụng đối với đúng loại biến đổi rất rõ (hình 8b): với ảnh bị dịch, accuracy tăng từ 72.1% lên 94.5% khi huấn luyện với phép dịch; xoay, co giãn, nhiễu Gauss và muối tiêu mỗi phép cải thiện cột tương ứng thêm từ 0.6 đến 1.8 điểm. Cả năm phép cùng lúc có mức tăng validation nhỏ nhất (+0.09), tức hiệu quả không cộng dồn.



Hình 8. Tăng cường dữ liệu cho MLP (trên, 3 seed) và CNN (dưới, 2 seed). (a) Hiệu validation accuracy so với điều kiện gốc cùng seed (trung bình ± độ lệch chuẩn của hiệu ghép cặp). (b) Accuracy test (%) khi ảnh test bị biến đổi mạnh; hàng là điều kiện huấn luyện, cột là loại biến đổi của tập test. Số liệu đầy đủ ở bảng 12.

CNN. Mọi điều kiện đều tăng validation accuracy (+0.17 đến +0.31 điểm) với độ lệch chuẩn ghép cặp từ 0.01 đến 0.20 điểm và accuracy test sạch tăng từ 98.82% lên 99.00–99.25%; "cả năm phép" có validation cao nhất và được chọn (test 99.25%). Chỉ có hai seed nên độ lệch chuẩn kém tin cậy, và vẫn có yếu tố dữ liệu gấp đôi, vì vậy đây là bằng chứng gợi ý chứ không phải xác lập. Khác với MLP, có một tác dụng bất lợi: huấn luyện chỉ với phép dịch hoặc chỉ với nhóm hình học làm accuracy trên ảnh bị nhiễu Gauss mạnh giảm từ 97.2% xuống 95.0% và 93.6%, và trên ảnh muối tiêu từ 97.5% xuống 96.0% và 95.0%; huấn luyện với chính các nhiễu này thì lên 98.3% và 98.7%, và với cả năm phép lên 98.9%. Một phép tăng cường có thể cải thiện loại biến đổi nó mô phỏng và làm giảm độ bền ở loại khác; ở thí nghiệm này chỉ ba điều kiện có nhiễu điểm ảnh (Gauss, muối tiêu, cả năm phép) không làm giảm cột nào so với điều kiện gốc.

Kết luận cho RQ4. Hiệu quả rõ và lặp lại được của tăng cường là độ bền với đúng loại biến đổi. Tác dụng lên độ chính xác trên ảnh sạch nhỏ hơn nhiễu giữa các seed ở MLP và nhất quán hơn nhưng chưa tách khỏi hiệu ứng lượng dữ liệu ở CNN.

6.3 Fashion-MNIST và EMNIST Digits (RQ5)

Fashion-MNIST [16] gồm ảnh 28×28 của 10 loại quần áo (60 000 train, 10 000 test). EMNIST Digits [4] có 240 000 ảnh train và 40 000 ảnh test chính thức; chúng tôi lấy một tập con phân tầng 60 000 ảnh từ phần train, chia 50 000/10 000 thành train/validation (cùng cỡ với MNIST) và giữ nguyên 40 000 ảnh test, đã xoay về chiều đứng như MNIST. Mỗi bộ dữ liệu có phép chia, hằng số chuẩn hóa và siêu tham số riêng, chọn bằng quét *rút gọn* một seed trên validation của bộ đó; kết quả được báo cáo riêng. EMNIST Digits không có baseline scikit-learn.

Thứ hạng giữa các mô hình khá ổn định ở hai bộ dữ liệu chữ số (bảng 6): Naïve Bayes $<$ softmax $<$ AdaBoost $<$ MLP $\leq$ CNN, với CNN cao hơn MLP 0.28 điểm trên 40 000 ảnh test EMNIST (khoảng tin cậy nhị thức 95% của mỗi accuracy rộng khoảng ± 0.1 điểm). Trên Fashion-MNIST thứ hạng thay đổi: MLP và CNN ngang nhau (88.59 và 88.49%, chênh 0.10 điểm, nhỏ hơn mọi nguồn nhiễu đã đo), AdaBoost chỉ hơn softmax 0.47 điểm, và Naïve Bayes giảm xuống 67.27%. AdaBoost cũng có khoảng cách train $-$ test lớn nhất (95.74% so với 84.74%, tức 11.0 điểm; MLP 4.85, CNN 2.66). Vì siêu tham số của hai bộ này chỉ được chọn bằng quét rút gọn và một seed, chúng tôi không kết luận mô hình nào tốt hơn khi chênh

Mô hình	MNIST	Fashion-MNIST	EMNIST Digits
Gaussian NB	81.32 / 81.16	67.27 / 66.03	80.03 / 80.13
Softmax regression	92.52 / 92.42	84.27 / 84.18	93.81 / 93.81
MLP	98.24 / 98.23	88.59 / 88.55	98.36 / 98.36
AdaBoost-SAMME	97.60 / 97.58	84.74 / 84.88	97.38 / 97.38
CNN	98.80 / 98.79	88.49 / 88.48	98.64 / 98.64

Bảng 6. Accuracy / macro-F1 trên tập test (%) của từng bộ dữ liệu; mỗi bộ dữ liệu có phép chia train/validation/test riêng, siêu tham số chọn riêng trên validation của bộ đó (quét rút gọn cho Fashion-MNIST và EMNIST Digits). Không so sánh chéo giữa các bộ dữ liệu.

Mô hình	Huấn luyện trên MNIST	Huấn luyện trên EMNIST Digits
Gaussian NB	49.80	80.03
Softmax regression	53.59	93.81
MLP	76.70	98.36
CNN	92.15	98.64
AdaBoost-SAMME	73.69	97.38

Bảng 7. Accuracy (%) trên 40 000 ảnh test của EMNIST Digits. Cột giữa là mô hình huấn luyện trên MNIST (dùng hằng số chuẩn hóa của MNIST) đánh giá trực tiếp; cột phải là mô hình huấn luyện trên EMNIST Digits. Đây là hai mô hình khác nhau.

lệch dưới khoảng 0.3 điểm.

Giới hạn của việc tổng quát hóa giữa các bộ dữ liệu. MNIST và EMNIST Digits cùng gán nhãn 0–9 cho chữ số, còn Fashion-MNIST gán 10 loại quần áo cho cùng các chỉ số; phép đánh giá một mô hình trên bộ dữ liệu có định nghĩa lớp khác vì vậy không có nghĩa và chương trình từ chối thực hiện. Phép kiểm tra chéo hợp lệ duy nhất là MNIST → EMNIST Digits (bảng 7), và cột giữa là các mô hình huấn luyện trên MNIST, khác với các mô hình huấn luyện trên EMNIST ở cột phải. Mọi mô hình đều giảm: Naïve Bayes từ 80.03% xuống 49.80%, softmax từ 93.81% xuống 53.59%, MLP từ 98.36% xuống 76.70%, AdaBoost từ 97.38% xuống 73.69%, và CNN từ 98.64% xuống 92.15%. CNN giảm ít nhất (6.5 điểm) và softmax nhiều nhất (40.2 điểm). Chúng tôi chưa tách riêng nguyên nhân; kiểu chữ, độ dày nét và cách căn giữa của hai bộ khác nhau có thể đóng vai trò, và thứ hạng này không cho phép suy ra khả năng tổng quát hóa sang chữ viết tay nói chung.

6.4 Bản chạy thử trên trình duyệt

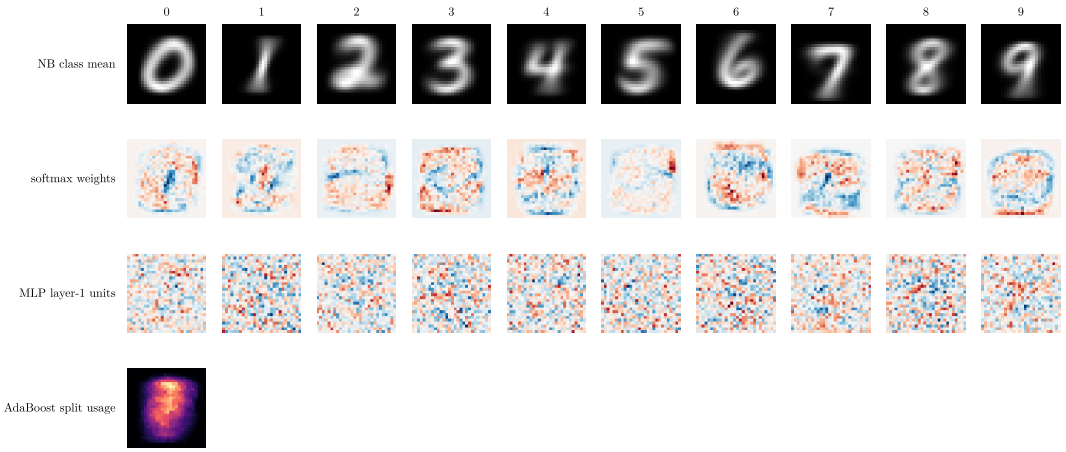
Mô hình của cả ba bộ dữ liệu được xuất ra ONNX (Naïve Bayes, softmax, MLP, CNN; bước chuẩn hóa nằm trong đồ thị) và một tệp nhị phân cho các cây của AdaBoost, rồi đối chiếu với mô hình Python: với 15 mô hình xuất ra, sai lệch xác suất lớn nhất giữa ONNX (hoặc bộ đánh giá nhị phân) và mô hình Python là 1.9×10^{-4} và độ trùng nhãn thấp nhất là 100.00% (trên 2 000 ảnh test cho mỗi mô hình). Bundle được đẩy lên Hugging Face Hub và *ghim theo mã commit* trong `web/model-lock.json`; trang web (React, TypeScript, ONNX Runtime Web [11]) chỉ tải đúng revision đó và kiểm tra SHA-256 từng tệp. Người dùng có thể vẽ hoặc chọn ảnh test, chọn bộ dữ liệu và một trong năm mô hình, rồi xem kết quả. Phần "nhìn vào bên trong" vẽ lại phép tính của *chính mô hình đã triển khai* (tích có trọng số của MLP, cửa sổ tích chập của CNN, log-likelihood của Naïve Bayes, các số hạng $x_i W_{ik}$ của softmax, các cây và phiếu của AdaBoost) từ metadata mà Python sinh ra, và đối chiếu với đầu ra của đồ thị ONNX. Phần này trực quan hóa *phép tính*; một bản đồ kích hoạt không chứng minh rằng pixel hay bộ lọc đó gây ra quyết định, nên nhóm không coi nó là lời giải thích quyết định của mô hình.

7 Thảo luận

7.1 RQ1 và RQ2: thứ hạng, độ phức tạp và chi phí

Dữ liệu cho thấy hai khoảng cách có bản chất khác nhau. Khoảng cách giữa Naïve Bayes, softmax và ba mô hình còn lại (từ 5.7 đến 17 điểm) lớn hơn mọi nguồn nhiễu đã đo và có thể gắn với giả thiết của từng mô hình. Khoảng cách giữa MLP, AdaBoost và CNN (dưới 1.2 điểm) cùng bậc với độ lệch chuẩn giữa các seed (0.29 điểm ở MLP), nên chỉ riêng thứ tự CNN trên MLP được các seed hỗ trợ.

Naïve Bayes. Hệ số tương quan trung bình 0.65 giữa hai pixel kề nhau trong cùng một lớp mâu thuẫn



Hình 9. Tham số hoặc thống kê của từng mô hình: ảnh trung bình theo lớp (Naïve Bayes), trọng số theo lớp (softmax; đỏ dương, xanh âm), các đơn vị ẩn đầu tiên của MLP, và tần suất pixel được dùng để chia nhánh (AdaBoost).

trực tiếp với giả thiết độc lập có điều kiện: khi các pixel tương quan, tổng log-likelihood của công thức (1) cộng cùng một thông tin nhiều lần. Bằng chứng gián tiếp là validation accuracy chỉ 56.05% ở mức làm trơn mặc định và 79.67% sau khi chỉnh `var_smoothing`, trong khi accuracy train và validation gần như trùng nhau (hình 3a): lỗi đến từ giả thiết của mô hình chứ không từ overfitting. Chúng tôi chưa làm thí nghiệm khử tương quan (ví dụ PCA trước Naïve Bayes) để xác nhận cơ chế này.

Softmax. Accuracy trên chính tập train chỉ đạt 93.62%, thấp hơn nhiều so với MLP (99.87%), tức là mô hình tuyến tính bị giới hạn bởi dung lượng (độ chệch cao) chứ không bởi sai khác giữa train và test. Lớp ẩn phi tuyến của MLP nâng test accuracy thêm 5.7 điểm, tức MLP biểu diễn được những biên quyết định mà siêu phẳng không biểu diễn được.

MLP và CNN. MLP có độ chệch thấp (train 99.87%) và khoảng cách 1.63 điểm đến test. CNN có ít tham số hơn 9.4 lần nhưng chính xác hơn và ít chênh giữa train và test; kết hợp với kết quả dịch ảnh ở hình 6b, điều này phù hợp với lập luận rằng chia sẻ trọng số và gộp cục bộ đưa vào một ràng buộc đúng với dữ liệu ảnh, nhưng thí nghiệm hiện tại không tách riêng từng thành phần.

AdaBoost. Accuracy tăng đơn điệu theo độ sâu cây và số vòng (hình 3d, hình 4c) và ensemble đạt 97.60% với các cây sâu tối đa 10. Cái giá cụ thể là 817.9 s huấn luyện, 18.705 ms mỗi ảnh và tệp 9.2 MiB (bảng 3); độ trễ này phụ thuộc vào cài đặt Python và không chỉ vào $O(Td)$. Chúng tôi không có baseline một cây đơn, nên không định lượng được phần lợi riêng của việc kết hợp 393 cây so với một cây sâu.

Khả năng giải thích. Hình 9 cho thấy các mô hình khác nhau ở mức độ có thể đọc được tham số. Ảnh trung bình theo lớp của Naïve Bayes là tham số μ_{kj} của chính mô hình; trọng số softmax là một khuôn mẫu trên mỗi lớp, đọc được ở mức mờ; các đơn vị ẩn đầu tiên của MLP gần như không đọc được bằng mắt; bản đồ tần suất chia nhánh của AdaBoost tập trung ở vùng trung tâm ảnh. Những hình này mô tả tham số hoặc thống kê của mô hình, không giải thích vì sao một ảnh cụ thể bị phân vào một lớp, và độ dễ đọc ở đây là nhận xét định tính, không phải một đại lượng đo được.

7.2 RQ3–RQ5: CNN, tăng cường dữ liệu và bộ dữ liệu khác

Thứ tự CNN trên MLP nhất quán qua các seed (mục 5) và qua hai bộ chữ số, nhưng không nhất quán trên Fashion-MNIST, nơi hai mô hình ngang nhau; lợi thế rõ nhất của CNN nằm ở độ bền với phép dịch và ở kích thước mô hình hơn là ở độ chính xác trên ảnh sạch. Tăng cường dữ liệu chủ yếu cải thiện độ bền với đúng loại biến đổi; tác dụng lên ảnh sạch chưa được tách khỏi hiệu ứng lượng dữ liệu, và với CNN việc chọn nhầm loại biến đổi có thể làm giảm độ bền với loại khác. Khi chuyển bộ dữ liệu, thứ hạng giữa các mô hình thay đổi ít giữa MNIST và EMNIST Digits nhưng nhiều hơn ở Fashion-MNIST, và việc đưa một mô hình ra ngoài bộ dữ liệu huấn luyện làm giảm accuracy từ 6.5 đến 40.2 điểm; chênh lệch giữa mô hình có cấu trúc không gian và mô hình không có là lớn nhất ở phép kiểm tra chéo đó.

8 Giới hạn và kết luận

Giới hạn. (1) *Số seed.* Mô hình chính chỉ được huấn luyện một lần (seed 42); độ lệch chuẩn giữa seed chỉ được ước lượng cho MLP (3 seed) và CNN (2 seed) trong thí nghiệm tăng cường, và chưa có cho Naïve

Mô hình	Độ chính xác	Thời gian huấn luyện và dự đoán	Khả năng giải thích
Naïve Bayes	Thấp nhất (81.32%): giả thiết độc lập bị mâu thuẫn bởi tương quan giữa các pixel kề nhau.	Rẻ: 0.4 s huấn luyện, 0.076 ms mỗi ảnh.	Tham số μ_{kj}, σ_{kj} là ảnh trung bình và phương sai theo lớp, đọc trực tiếp.
Softmax	92.52%; train chỉ 93.62%, nên bị giới hạn bởi dung lượng của biên tuyến tính.	Nhanh khi dự đoán (0.010 ms mỗi ảnh); huấn luyện 12.7 s.	Trọng số theo lớp vẽ được thành ảnh; $x_i W_{ik}$ phân rã chính xác điểm số.
MLP	98.24%; chênh train–test 1.63 điểm.	11.0 s huấn luyện, 0.042 ms mỗi ảnh.	Khó: đơn vị ẩn gần như không đọc được bằng mắt; 535 818 tham số.
AdaBoost-SAMME	97.60%, gần MLP; train 100%.	Đắt nhất: 817.9 s huấn luyện, 18.705 ms mỗi ảnh (393 cây, vòng lặp Python).	Bản đồ tần suất pixel được cây dùng; điểm cộng dồn là tỉ lệ phiếu, chưa phải xác suất.
CNN	Cao nhất (98.80%) với 56 714 tham số; bền hơn khi ảnh bị dịch.	143.0 s huấn luyện (CPU), 0.315 ms mỗi ảnh.	Bản đồ kích hoạt cho biết một lớp tính ra gì, không chứng minh nguyên nhân của quyết định.

Bảng 8. Tóm tắt ưu và nhược điểm của năm mô hình theo bốn tiêu chí của đề bài. Số liệu lấy từ bảng 2 và 3; nhận xét về khả năng giải thích là định tính (hình 9).

Bayes, softmax hay AdaBoost (có tính ngẫu nhiên riêng ở việc chọn đặc trưng). (2) *Tìm siêu tham số.* Quét từng tham số không xét tương tác; với AdaBoost, độ sâu tốt nhất (10) là giá trị lớn nhất trong khoảng $\{2, 4, 6, 8, 10\}$, và với CNN, số kênh tốt nhất $(16, 32)$ cũng là giá trị lớn nhất trong ba giá trị đã thử, nên hai mô hình này có thể chưa được tinh chỉnh tới mức tối ưu; MLP được thử nhiều cấu hình hơn (22 so với 14 và 10). (3) *Tăng cường dữ liệu.* Thiếu điều kiện đối chứng có cùng lượng dữ liệu nhưng không biến đổi, nên không tách được ảnh hưởng của dữ liệu thêm; CNN chỉ có hai seed. (4) *Thời gian.* Đo trên một máy, chỉ CPU, có thể trùng với công việc khác; cùng cấu hình CNN có thời gian huấn luyện từ 143.0 s đến 312 s. (5) *Bộ dữ liệu khác.* Fashion-MNIST và EMNIST Digits dùng quét rút gọn một seed, EMNIST không có baseline scikit-learn, và chưa thử bộ chữ số viết tay tiếng Việt. (6) *Bản chạy thử.* Nét vẽ trên màn hình có phân phối khác MNIST và nhóm không có dữ liệu có nhãn để đo độ chính xác của đường vẽ; chỉ có độ chính xác trên ảnh test thật.

Kết luận. Trên MNIST với cùng giao thức, CNN đạt accuracy test cao nhất (98.80%) với ít tham số hơn MLP gần mười lần, MLP đạt 98.24% và AdaBoost 97.60% với thứ tự chưa chắc chắn giữa hai mô hình này, softmax đạt 92.52% và Naïve Bayes 81.32%. Các cài đặt tự viết khớp scikit-learn trong phạm vi các khác biệt đã giải thích (bảng 4). Hai việc cần làm để củng cố các kết luận chưa chắc chắn là chạy nhiều seed cho AdaBoost và cho các mô hình chính, và thêm điều kiện đối chứng cùng lượng dữ liệu cho thí nghiệm tăng cường.

9 Phân công công việc và công bố sử dụng AI

Phân công. Bảng dưới đây chỉ ghi những gì kiểm chứng được từ lịch sử Git của kho mã (`git shortlog`, các pull request #1–#4 và `WORKFLOW.md`); tỉ lệ đóng góp do hai thành viên tự thống nhất và xác nhận, không suy ra từ số commit.

Công bố sử dụng AI. Theo phụ lục của đề bài, nhóm công bố rằng phần hoàn thiện đồ án sau commit `f9a602b` được viết với sự hỗ trợ của trợ lý lập trình AI Claude Code (Anthropic): `SoftmaxRegression`, `AdaBoostSAMME` và cây quyết định có trọng số, các sửa đổi cho `MLP`, độ đo, tiền xử lý và `Evaluator`, bộ kiểm thử, pipeline thực nghiệm và tích hợp Weights & Biases, CNN, các thí nghiệm mở rộng, quy trình xuất ONNX và Hugging Face, website và trình khám phá, notebook tái lập, mã LaTeX của báo cáo và slide. Trợ lý đã chạy mã, kiểm thử và thí nghiệm trong môi trường của nhóm; các con số trong báo cáo được sinh từ tệp kết quả do chương trình ghi. Các thành viên chịu trách nhiệm về mã nộp và phải đọc, hiểu và giải thích được nó. Mức hỗ trợ của AI ở giai đoạn trước commit `f9a602b`: **[Nhóm cần tự xác nhận và điền: có hay không dùng công cụ AI, công cụ nào, cho phần nào]**. Các tài liệu tham khảo được kiểm tra thủ công về sự tồn tại và thông tin thư mục.

Tài liệu tham khảo

[1] Lukas Biewald. Experiment tracking with Weights and Biases, 2020. Software available from `https://www.wandb.com/`.

Thành viên	MSSV	Công việc có bằng chứng trong kho mã	Tỉ lệ
Lâm Nhật Tiến (GitHub: Tien-Lam28)	24280027	23 commit (13/10–06/11/2026) và PR #1–#4: dựng kho, README và `WORKFLOW.md`, tải và tiền xử lý dữ liệu, lớp cha `BaseClassifier`, độ đo, `Evaluator`, K-fold, Gaussian NB, bản đầu của MLP, EDA, notebook trực quan; được phân công Naïve Bayes và MLP.	50%
Trần Trinh Nhân (GitHub: TranTrinhNhan1)	24280088	Khởi tạo kho; được phân công mô hình tuyến tính, ensemble và phần mở rộng. Giai đoạn hoàn thiện (softmax, AdaBoost-SAMME, tái cấu trúc MLP và độ đo, bộ kiểm thử, pipeline thực nghiệm, CNN, các thí nghiệm mở rộng, xuất mô hình, website, báo cáo, slide) được thực hiện cùng trợ lý AI như công bố dưới đây.	50%

Bảng 9. Phân công và tỉ lệ đóng góp.

[2] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[3] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth, 1984.

[4] Gregory Cohen, Saeed Afshar, Jonathan Tapson, and André van Schaik. EMNIST: an extension of MNIST to handwritten letters, 2017. arXiv:1702.05373.

[5] Thomas G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923, 1998.

[6] Pedro Domingos and Michael Pazzani. On the optimality of the simple Bayesian classifier under zero-one loss. *Machine Learning*, 29(2–3):103–130, 1997.

[7] Richard O. Duda, Peter E. Hart, and David G. Stork. *Pattern Classification*. Wiley, 2 edition, 2001.

[8] Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.

[9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.

[10] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[11] ONNX Runtime developers. ONNX Runtime, 2021. https://onnxruntime.ai/.

[12] Lutz Prechelt. Early stopping – but when? In *Neural Networks: Tricks of the Trade*, volume 1524 of *Lecture Notes in Computer Science*, pages 55–69. Springer, 1998.

[13] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[14] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.

[15] Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, volume 28 of *PMLR*, pages 1139–1147, 2013.

[16] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms, 2017. arXiv:1708.07747.

[17] Ji Zhu, Hui Zou, Saharon Rosset, and Trevor Hastie. Multi-class AdaBoost. *Statistics and Its Interface*, 2(3):349–360, 2009.

A Cấu hình được chọn

Bảng 10 liệt kê cấu hình được chọn của mọi mô hình. Toàn bộ các lượt quét (kèm khoảng cách train–validation và thời gian huấn luyện mỗi cấu hình) nằm trong thư mục `results/` (`tuning` và `figures`).

Mô hình	Siêu tham số được chọn	Số cấu hình đã chạy
Gaussian NB	`var_smoothing=0.1`	12
Softmax regression	`learning_rate=0.03, l2=0.0, epochs=67, batch_size=128`	16
MLP	`hidden_sizes=(512, 256), learning_rate=0.05, momentum=0.9, p_drop=0.0, batch_size=128, epochs=40`	22
AdaBoost-SAMME	`n_estimators=393, max_depth=10, learning_rate=0.1, n_bins=32, max_features=0.5`	14
CNN	`channels=(16, 32), kernel_size=3, dense=(64,), learning_rate=0.01, momentum=0.9, p_drop=0.2, batch_size=128, epochs=20`	10

Bảng 10. Cấu hình chọn bằng validation. `epochs` của softmax và `n_estimators` của AdaBoost được lấy từ đường cong validation của một lần chạy dài.

B Chỉ số đầy đủ trên ba tập

Bảng 11 đưa các chỉ số macro trên cả train, validation và test; bảng 2 chỉ giữ chỉ số test để dễ đọc. Ma trận nhầm lẫn của cả ba tập (không chỉ test, như ở hình 2) được lưu trong `results/final/*.json`.

	Train				Validation				Test			
Mô hình	Acc	P	R	F1	Acc	P	R	F1	Acc	P	R	F1
Gaussian NB	80.13	81.85	79.74	79.98	79.67	81.39	79.27	79.50	81.32	82.98	80.95	81.16
Softmax regression	93.62	93.56	93.53	93.54	92.15	92.07	92.05	92.05	92.52	92.45	92.41	92.42
MLP	99.87	99.87	99.87	99.87	98.18	98.17	98.17	98.17	98.24	98.23	98.23	98.23
AdaBoost-SAMME	100.00	100.00	100.00	100.00	97.59	97.59	97.57	97.58	97.60	97.58	97.58	97.58
CNN	99.40	99.40	99.39	99.39	98.65	98.65	98.63	98.64	98.80	98.79	98.79	98.79

Bảng 11. Accuracy, precision, recall và F1 macro (%) của năm mô hình trên MNIST, một seed (42).

C Tăng cường dữ liệu: số liệu tổng hợp

	MLP (3 seed)		CNN (2 seed)	
Điều kiện	Δ val (điểm)	Test (%)	Δ val (điểm)	Test (%)
Không tăng cường	–	98.02	–	98.82
Xoay	+0.57 ± 0.47	98.48	+0.23 ± 0.01	99.17
Dịch chuyển	+0.54 ± 0.55	98.41	+0.22 ± 0.03	99.17
Co giãn	+0.42 ± 0.45	98.33	+0.20 ± 0.01	99.06
Nhiễu Gauss	+0.38 ± 0.42	98.35	+0.19 ± 0.20	99.06
Nhiễu muối tiêu	+0.37 ± 0.40	98.32	+0.17 ± 0.06	99.00
Hình học (xoay, dịch, co giãn)	+0.31 ± 0.33	98.24	+0.25 ± 0.16	99.20
Cả năm phép	+0.09 ± 0.40	98.11	+0.31 ± 0.01	99.25

Bảng 12. Tăng cường dữ liệu với cấu hình đã chọn: Δ là hiệu validation accuracy so với điều kiện gốc *cùng seed* (trung bình ± độ lệch chuẩn mẫu của hiệu), "Test" là accuracy test sạch trung bình trên các seed. Mỗi điều kiện tăng cường dùng 100 000 ảnh train, điều kiện gốc 50 000.

D Tái lập kết quả

Môi trường dùng `uv` (Python ≥ 3.11). Mỗi lệnh nhận thêm `-dataset fashion_mnist` hoặc `-dataset emnist_digits`. Notebook `notebooks/reproduce_results.ipynb` chạy lại các bước chính trên tập con nhỏ.

```
uv sync --extra track --extra export
uv run python -m src prepare            # tải dữ liệu, chia train/val, chuẩn hóa
uv run python -m src tune --model all   # quét siêu tham số -> results/tuning
uv run python -m src train --model all  # huấn luyện cuối, đánh giá 3 tập
uv run python -m src baselines          # scikit-learn, cùng siêu tham số
uv run python -m src augment --model mlp; uv run python -m src augment --model cnn --seeds 2
uv run python -m src compare            # CNN so với MLP, kiểm tra chéo
uv run python -m src report             # bảng, hình, đoạn LaTeX
uv run python -m src export --site      # ONNX + kiểm tra số học
```